

noCommerce OpenTelemetry

Order Placement Flow



Flow chosen: customer places an order

Highest operational value, revenue-critical path.

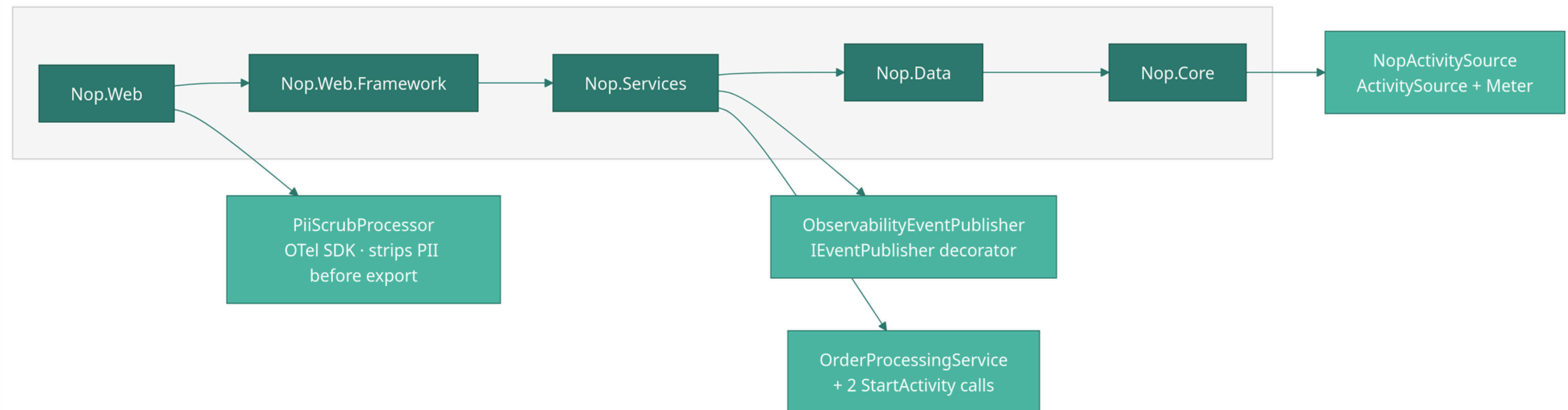
If this breaks, no money moves.

Architecture & Instrumentation

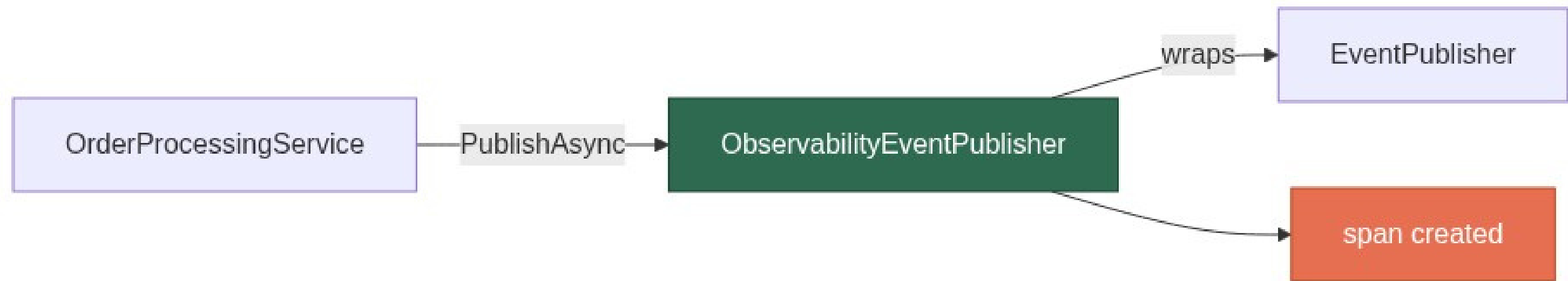
- Production-grade ASP.NET Core e-commerce - 9k+ GitHub stars
- Events via `IPublisher` - all business events through one interface
- Zero OTel out of the box

What was added:

- `NopActivitySource`: shared `ActivitySource` + `Meter`
- `ObservabilityEventPublisher`: `IPublisher` decorator
- `PiiScrubProcessor`: strips PII at SDK level
- `OrderProcessingService`: +2 `StartActivity` calls



Key Finding: IEventPublisher



Every business event in the system flows through one method: *IEventPublisher.PublishAsync<T>*

Decorate it once. Capture everything.
Zero changes to service code

Privacy Strategy Overview

Problem: ProcessPaymentRequest carries CreditCard* fields.

Solution: PiiScrubProcessor - a BaseProcessor<Activity> at SDK level

- Runs after spans are created, before they reach the OTLP exporter
- Strips: *CardNumber, CardCvv2, CardName, Email, FirstName, LastName, PhoneNumber, Address1, CustomerIp*

Why at SDK level?

- Data never leaves the process
- Centralised: individual instrumentation points don't need to remember what to exclude

Custom Metrics Overview

order.placement.duration (histogram, ms)

- Measures end-to-end order placement time
- Tagged with `success=true|false`
- *Why*: rising p95 signals checkout pipeline degradation before users see errors

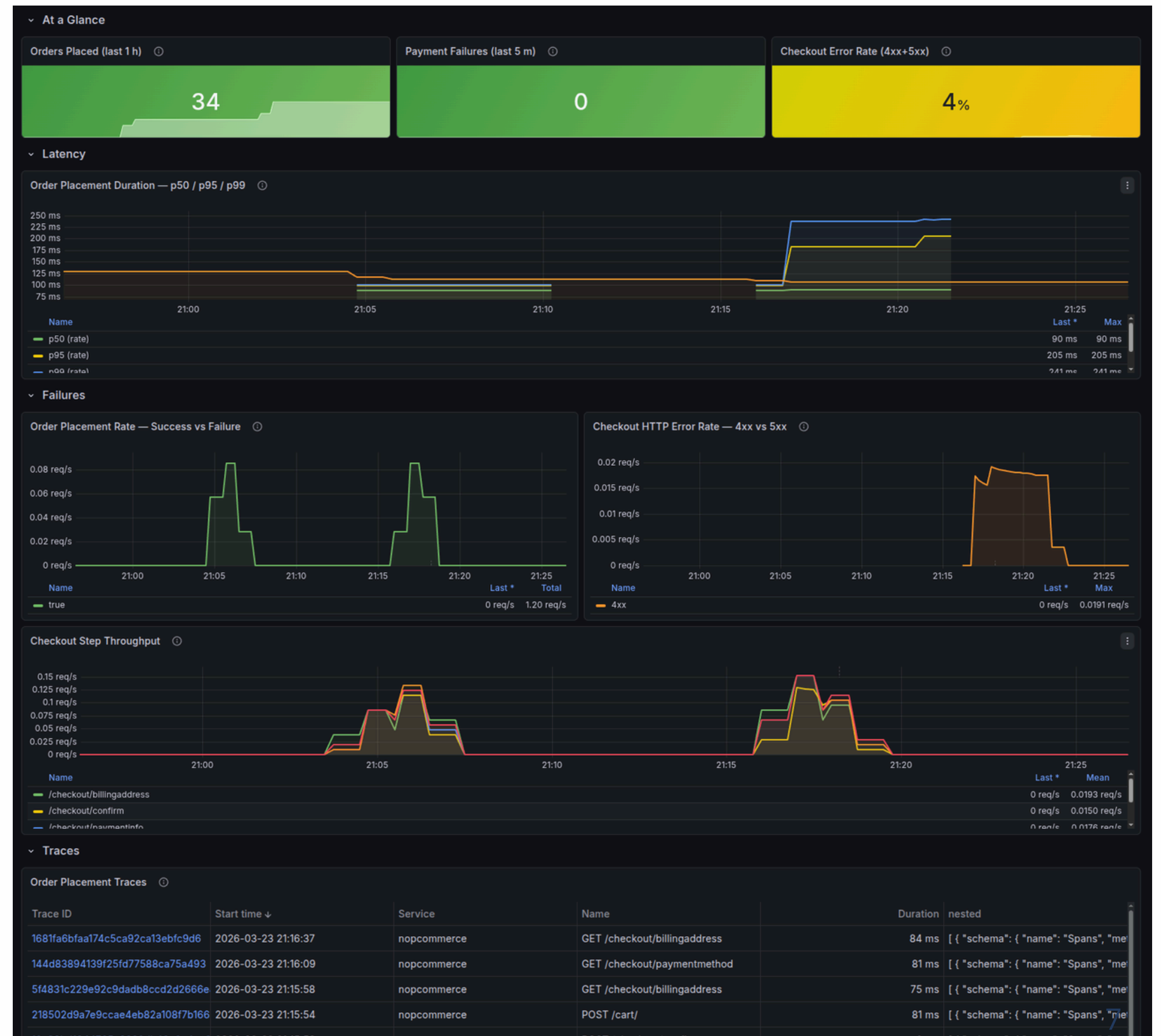
order.payment.failures (counter)

- Counts payment processing failures, tagged by error reason
- *Why*: distinguishes gateway outage from declined cards (varied) - an on-call engineer can act on this immediately

Grafana Dashboard Insights

8 panels

- Orders placed
- Payment failures
- p50/p95/p99 latency
- Success vs failure rate
- Step throughput
- Distributed traces



Load Test Performance Results

Scenario	VUs	Orders	Success	p(95) HTTP
smoke	1	2	100%	90 ms
demo	3 good + 1 bad	9 good / 8 bad	53% (intentional)	93 ms
load	10	215	100%	109 ms
stress	up to 50	875	100%	110 ms

- 14955 HTTP requests (stress): **0% failure rate**
- demo: 2.57% HTTP failure rate - antiforgery 400s from bad VU

Reading p95: 95% of HTTP requests complete below this value.
p50 = typical / p95 = near-worst case

Design Critique

Got right:

IEventPublisher: one decorator, full coverage

Layered arch: no surprise cross-layer calls

ASP.NET Core: HTTP spans for free

Got wrong:

Runtime DI breaks standard decorator patterns

ProcessPaymentRequest mixes PII + operational data

Zero OTel scaffolding: observability as afterthought